ChatGPT

وثيقة تحليل متطلبات RESTful API باستخدام Laravel (15 يوليو 2026)

ملخص تنفيذي

هذه الوثيقة تقدم خطة عملية لتحليل وتوثيق متطلبات مشروع **RESTful API** باستخدام Laravel، بناءً على طلب عميل محتمل. تشتمل الوثيقة على قائمة أسئلة استيضاحية تفصيلية للعميل، وتقسيم المتطلبات إلى ميزات واضحة وقابلة للقياس مع معايير قبول لها، بالإضافة إلى تقدير زمني مفصل، وتحديد المخاطر ونقاط القرار التقنية. كما توضح مخرجات التسليم النهائية (ملفات التوثيق والأدوات). أخيراً، تشمل خطوات الاتصال والتسليم المرحلية ومخططات مرئية (ER, Flow, Gantt). تضمن الوثيقة توضيح الافتراضات غير المحددة ومراعاة المصداقية مع المصادر الرسمية (مثل وثائق Laravel وOpenAPI وAWS) وتاريخ المراجعة الحالي 15 يوليو 2026.

الهيكل العام للمستند

- **1. جمع المعلومات (أسئلة للعميل)**
- **1. تفكيك المتطلبات إلى ميزات**
- **1. القيود والافتراضات المفتوحة**
- **1. تقدير الجهد الزمني والمهام**
- **1. المخاطر ونقاط القرار التقنية/المنتجية**
- **1. المخرجات النهائية المقترحة**
- **1. خطة التواصل والتسليم المرحلية**
- **1. جداول ومخططات مرئية (ER, Flow, Gantt)**
- **1. المصادر المفضلة**
- **1. صيغة الملف النهائي للتسليم وأدوات التحويل**

يرجى مراجعة الهيكل أعلاه وتأكيده قبل المضي في كتابة التفاصيل النهائية وإنتاج ملف PDF مع الرسومات.

١. جمع المعلومات (أسئلة للعميل)

لضمان فهم كامل لاحتياجات العميل وتفاصيل النظام، نطرح الأسئلة التفصيلية التالية:

- **هدف النظام:** ما هي المشكلة الرئيسية التي يحلها الـ API؟ وما المنافع التجارية المتوقعة؟
- **الفئات المستهدفة من المستخدمين والأدوار:** من هم المستخدمون (عملاء موبايل، مسؤولون، زوار، إلخ)؟ ما صلاحيات كل دور؟
- **حالات الاستخدام الأساسية:** ما العمليات الرئيسية المطلوبة؟ (مثلاً: تسجيل/تسجيل دخول، CRUD على الكيانات Business logic، تقرير).
- **نماذج البيانات (الكيانات):** ما الكيانات الأساسية (مثل Users, Posts, Products) والعلاقات بينها؟
- **أحجام البيانات المتوقعة:** كم عدد المستخدمين المتوقع؟ حجم المعاملات شهرياً أو سنوياً؟ أي قيود على التخزين أو سرعة الوصول؟
- **متطلبات الأداء:** ما زمن الاستجابة المستهدف (مثلاً <200ms للقراءات)؟ هل هناك توقع ذروة في الحمل؟
- **الأمان والمصادقة:** هل الـ API علني أم يحتاج تسجيل دخول؟ ما طريقة المصادقة المقترحة (Bearer Token, OAuth2, JWT)؟ مدى حساسية البيانات المخزنة/المعالجة؟
- **الصلاحيات وإدارة الوصول (Authorization):** هل هناك قيود خاصة على بعض العمليات (مثلاً فقط Admin يمكنه الحذف)؟ ما سياسات الوصول المطلوبة (Policies)؟
- **رفع الملفات والصور:** هل يتضمن المشروع وظائف رفع ملفات أو صور؟ ما الأنواع والأحجام المتوقعة؟ أين سيتم تخزينها (S3/خادم محلي)؟
- **التكاملات الخارجية:** هل يحتاج النظام للتكامل مع خدمات أو APIs خارجية (بوابة دفع، خدمة بريد إلكتروني، خرائط، إلخ)؟

- **توثيق الـ API**: هل يطلب العميل مواصفات API (Swagger/OpenAPI) أو Postman Collection؟ هل تتوقع وجود مستندات HTML أو PDF للمطورين؟
- **بيئات النشر:** هل سيتم النشر على خوادم داخلية (on-premise) أم سحابية (AWS، Azure، إلخ)؟ هل Docker/VPN مطلوب؟
- **القيود القانونية/الخصوصية:** هل هناك امتثال لمعايير معينة (مثل GDPR للبيانات الشخصية) أو قيود تنظيمية (PCI-DSS للدفع)؟
- **الميزانية والجدول الزمني:** ما المهلة الزمنية المطلوبة لتسليم المشروع والميزانية المتاحة؟ هل هناك مواعيد نهائية مرحلية معتمدة؟

توضيح: هذه الأسئلة تغطي الجوانب الأساسية. قد نضيف أو نعدل بعض الأسئلة حسب مجال العميل ونتائج اجتماع الاكتشاف الأولي.

٢. تفكيك المتطلبات إلى ميزات

نحول متطلبات العميل إلى **ميزات (Features)** واضحة، مع تحديد المدخلات والمخرجات، ومعايير القبول الخاصة بكل ميزة، وترتيب أولوياتها (Must/Should/Could). الجدول التالي يوضح مثالاً تخيلياً (القيم مجرد نموذج):

الميزة	الوصف	المدخلات	المخرجات	معايير القبول	الأولوية
التسجيل / المصادقة	تمكين المستخدمين من التسجيل وتسجيل الدخول عبر البريد الإلكتروني.	معلومات المستخدم (البريد وكلمة المرور)	رمز وصول (JWT) أو جلسة مصادق عليها	- إنشاء مستخدم جديد وإرجاع رسالة نجاح. - تسجيل الدخول الصحيح يعيد رمز JWT صالح. - فشل المطابقة يرجع خطأ مصادق مناسب.	أساسية (Must)
إدارة المستخدمين (CRUD)	عمليات إنشاء/قراءة/تحديث/حذف بيانات المستخدمين (Admin فقط يمكنه التعديل/الحذف).	بيانات المستخدم (email, الاسم)	كيان مستخدم أو قائمة المستخدمين	- Admin فقط يمكنه إنشاء أو تعديل أو حذف حسابات المستخدمين. - جميع الاستجابات بصيغة JSON مع بيانات المستخدم.	أساسية (Must)
إدارة المنشورات (CRUD)	إنشاء وتحرير وحذف وعرض المنشورات (عناوين، محتوى، صور).	عنوان المنشور، المحتوى (وصف)، (صورة اختيارية)	كيان منشور أو قائمة منشورات	- المنشور يتطلب عنوان ومحتوى. إذا ناقصة، يتم إرجاع خطأ تحقق. - المستخدم فقط صاحب المنشور أو Admin يمكنه تعديل/حذف. - **عرض المنشورات** يدعم الترقيم (pagination) ويرجع JSON منسّق.	مهمة (Should)
التعليقات	إضافة تعليقات إلى المنشورات وعرضها.	معرف المنشور ونص التعليق	كيان تعليق جديد أو قائمة تعليقات	- فقط المستخدمين المسجلين يمكنهم التعليق. - حذف/تعديل التعليق يحق لصاحبه أو Admin فقط. - الردود بصيغة JSON تشمل معلومات المعلق وتاريخ الإنشاء.	اختيارية (Could)
رفع الملفات	رفع ملفات/صور للمستخدمين أو المنشورات (jpg/png/pdf).	ملف (صورة أو مستند)	رابط تحميل الملف المخزن	- قبول أنواع ملفات محددة وحجم أقصى (مثلاً 5MB). - يتم تخزين الملف على S3 أو محلي (حسب الإعداد)، وحذف القديم عند التحديث. - الرد JSON يتضمن رابط وصول آمن للملف.	مهمة (Should)

الميزة	الوصف	المدخلات	المخرجات	معايير القبول	الأولوية
البحث / التصفية	البحث في المنشورات أو المستخدمين بكلمات مفتاحية أو تصنيفات.	معايير (نص، تاريخ)	نتائج منقحة حسب المعايير	- دعم البحث النصي (like) وتصفية بالحقل الزمني أو الخاصيات. - النتائج مفصولة صفحات وترجع مع بيانات الميتا (مثل إجمالي الصفحات).	اختيارية (Could)
توثيق الـ API	توليد وثائق API أوتوماتيكية قابلة للمراجعة (مثلاً Scribe).	التعليقات التعريفية أو نقاط النهاية في الكود	ملفات التوثيق (HTML) وOpenAPI وPostman	- توليد ملف OpenAPI (JSON/YAML) تلقائياً. - توليد مجموعة Postman JSON لاستيرادها. - واجهة HTML تفاعلية للوثائق (Try It Out).	اختيارية (Could)
تكاملات خارجية	واجهات إلى خدمات خارجية (مثل بوابة دفع أو خدمات بريد).	مفاتيح التكامل وطلبات خارجية	استجابات الخدمات أو ملخص العمليات	- يمكن للـ API استدعاء واجهات خارجية بأمان والتعامل مع أخطاء الطرف الثالث. - وثائق تضم وصف التكامل وكيفية التجربة (مثلاً Webhook endpoint).	اختيارية (Could)

ملاحظة: - معايير القبول: شروط يجب تحقيقها اعتباراً من متطلبات العميل.
- **الأولويات (Must/Should/Could):** لتوجيه العمل (ضروري/مهم/اختياري).

٣. القيود والافتراضات المفتوحة

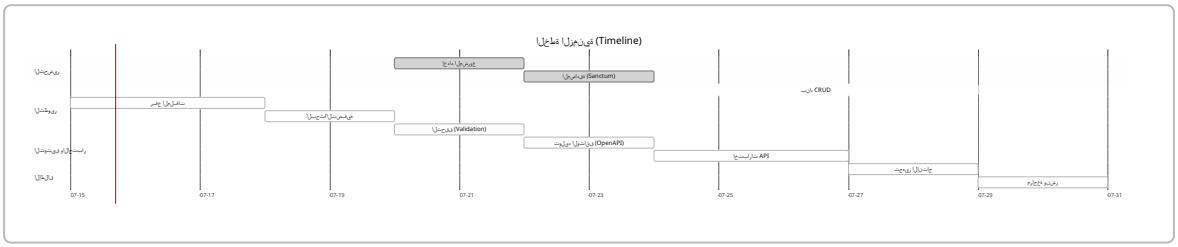
- **قيود تقنية:** استخدام Laravel (إصدار حديث)، قاعدة بيانات SQL (MySQL/PostgreSQL)، Docker ضمن خطة النشر، واجهة برمجة تطبيقات stateless.
- **قيود غير تقنية:** موارد محدودة (فريق 1-2 مطورين)، وإمكانية رفع متطلبات لاحقة فقط في النسخ المستقبلية (تنفيذ مبدئي محدود بتوقيت محدد).
- **افتراضات عامة:** افترضنا أن المشروع لا يتضمن اتصالاً خارجيًا مع OAuth اجتماعي (مثل تسجيل Google) ما لم يُذكر. إذا كان العميل يتوقع ميزات حماية خاصة (GDPR مثلاً)، يجب توضيحها. كما يفترض عدم وجود قاعدة بيانات ضخمة في البداية (يمكن لاحقاً ترقية التخزين).
- **الافتراضات غير محددة:** لم يحدد العميل نطاق الرسوم البيانية (عدد الـ API endpoints) أو عدد الاستخدامات الحرجة، لذا نفترض سيناريو متوسط الضغط. كذلك لم يُوَضح نوع التطوير المستمر المطلوب بعد الإطلاق، فسنفترض تسليم النسخة الأساسية مع صلاحيات صيانة لاحقة.

٤. تقدير الجهد الزمني والمهام

نقسم المشروع إلى مهام رئيسية مع تقدير زمني تقريبي (بالأيام عمل) لفريق مكوّن من مطورين 1-2:

المهمة	الوصف	التقدير (أيام عمل)	الملاحظات
إعداد المشروع الأساسي	إنشاء مشروع Laravel جديد، ضبط بيئة التطوير، إنشاء قواعد البيانات، إعداد `.env`	2	بداية المشروع
نظام المصادقة	تركيب Sanctum/Passport، إعداد جداول المستخدمين والجلسات	2	بعد إعداد المشروع
بناء CRUD الأساسي	تطوير وظائف CRUD للمستخدمين ثم المنشورات والتعليقات	5	بعد المصادقة

المهمة	الوصف	التقدير (أيام عمل)	الملاحظات
نظام رفع الملفات	إعداد S3 أو التخزين المحلي للملفات، وتطوير رفع الصور/ملفات	3	متزامن مع CRUD المنشورات
البحث والترشيح	تطوير ميزات البحث/فلترة للمنشورات والمستخدمين	2	بعد الوظائف الأساسية
التحقق (Validation)	كتابة Form Requests وفحص البيانات الواردة	2	متزامن مع بناء الـ CRUD
توثيق الـ API	توليد مواصفات OpenAPI (Swagger) ومجموعة Postman (Scribe)	2	بعد تحديد endpoints
كتابة اختبارات API	تطوير اختبارات Feature بإطار PHPUnit/Pest	3	متوازي مع تطوير الميزات
إعداد الإنتاج (Release)	تهيئة بيئة الإنتاج (config، تنصيب الخادم، إعداد النسخ الاحتياطية)	2	نهاية المشروع
مراجعة نهائية ونشر	اختبار حمل، مراجعات أمنية، نشر النسخة النهائية	2	تسليم نهائي



ملاحظة: هذه تقديرات أولية. يُعاد تقييمها بعد كل مرحلة رئيسية لضبط الجدول حسب التطورات الحقيقية.

٥. المخاطر ونقاط القرار التقنية/المنتجية

- **مصادقة Sanctum vs Passport:** Sanctum خفيف ومناسب لتطبيقات Single-Page أو أولية ببساطة التوكن 【L962†14-L967】 . Passport (OAuth2) مناسب إذا كان النظام يحتاج دعم OAuth كامل أو مشاركة مع مطورين خارجيين.
- **مونوليث مقابل مايكروسيرفس:** الهندسة المونوليثية (التقليدية) أبسط للتطوير والنشر (قاعدة شفرة واحدة ونشر واحد) 【L462†34-L470】 ، بينما المايكروسيرفس تمنح مرونة واستقلالية أكبر لكل خدمة 【L547-L555†34】 على حساب زيادة التعقيد والتنسيق. نميل لبدء مشروع متكامل (مونوليث) ومن ثم التفكير في تقسيمه إذا دعت الحاجة للنطاق الكبير.
- **التخزين: S3 مقابل التخزين المحلي:** S3 يمنح قابلية توسع عالية وموثوقية (11 تسعة للثبات) 【L75-L80†21】 ودعم تحميل ملفات بحجم كبير. التخزين المحلي أسهل في البداية لكن محدود بموارد السيرفر ويتطلب إدارة احتياطية يدوية. نرشح S3 للمشاريع ذات حجم الملفات الكبير 【L68-L72†21】 .
- **توثيق API: Scribe مقابل Swagger:** Scribe (حزمة Laravel) تولد التوثيق تلقائياً (HTML, OpenAPI, Postman) من الكود 【L54-L58†12】 مما يوفر وقتاً. كتابة Swagger/YAML يدوياً يمنح تحكماً كاملاً ولكنه يأخذ وقتاً. ننصح ببدء Scribe لضمان معايير وتحديث تلقائي.
- **إصدار API (Versioning):** من الأفضل اعتماد مسار إصدار واضح (مثلاً `/api/v1/...`) لضمان التوافق عند التحديث. يسهل هذا تقديم نسخ جديدة دون كسر عمل العملاء الحاليين.
- **قضايا CORS والأمان:** ضبط رؤوس CORS للسماح بالنطاقات الموثوقة ضروري للتكامل مع الواجهات الأمامية. وبما أن النظام يستعمل التوكن (Stateless)، فلا حاجة لـ CSRF التقليدي.
- **حماية الخادم (Rate Limiting):** يُفضل ضبط حدود للطلبات per IP أو per مستخدم لمنع إساءة الاستخدام، خاصة لو كان متوقع حمل عالي. يمكن استخدام ميزات Laravel (throttle) للحماية.
- **تكاملات خارجية:** إذا كان هناك تكامل مع خدمات خارجية، يجب تحديد كيفية إدارة المفاتيح السرية (env) والتعامل مع أخطاء الاستجابة من تلك الخدمات (مثل إعادة المحاولة أو fallback).

٦. المخرجات النهائية المقترحة

عند الانتهاء، يتم تسليم مجموعة ملفات ومخرجات تسهل على العميل والفريق استخدام الـ API:

- **مواصفات الـ API (OpenAPI Spec):** ملف `openapi.yaml` أو `openapi.json` يتبع معيار OpenAPI (يُفضّل YAML)【32†L243-L245】.
- **Postman Collection (JSON):** ملف `collection.json` يجمع نقاط النهاية والأمثلة لتجربة الـ API باستخدام Postman【30†L124-L131】.
- **مخطط العلاقات (ER Diagram):** ملف صورة (PNG/SVG) يظهر الكيانات والعلاقات في قاعدة البيانات، لسهولة الفهم.
- **خارطة نقاط نهاية الـ API (Flowchart):** مُخطط يوضح تدفق الطلبات بين المستخدمين والخوادم، مفيد للتواصل الداخلي.
- **دليل النشر (Deployment Guide):** ملف PDF/Markdown يشرح خطوات إعداد الخادم أو البيئة السحابية ونشر التطبيق (متغيرات البيئة، إعدادات S3، إلخ).
- **اختبارات الـ API (Test Suite):** ملفات الاختبارات (PHPUnit/Pest) التي تغطي الوظائف الأساسية.
- **وثائق (Scribe (HTML):** موقع توثيق تفاعلي شامل للـ API نتاج تشغيل Scribe، يتضمن أمثلة وتجربة "Try It Out".
- **المخرجات الخاصة المطلوبة:** تنفيذ طلب العميل "صدرلي النتيجة باستخدام الأدوات المناسبة على شكل ملف PDF مع الرسومات"؛ أي تسليم نسخة من هذا التقرير بصيغة PDF تتضمن جميع الرسومات (ER, Flow, Gantt).
- **مستودع الكود/الوثائق:** هيكل مشروع على GitHub يحوي الشفرة الكاملة وملفات التوثيق (Markdown).

قائمة الملفات المسلمة للعميل: `openapi.yaml` , `postman_collection.json` , `er_diagram.png` , `api_flowchart.png` , `deployment_guide.pdf` (أو .md), `test_suite.php` , `documentation.html` , **والنسخة النهائية** `report.pdf` (الناتج من Markdown).

٧. خطة التواصل والتسليم المرحلية

1. **مرحلة الاكتشاف الأولي:**

2. **الإجراء:** عقد اجتماع أو مكالمة مع العميل لمراجعة الأسئلة في القسم ١ وجمع الإجابات.

3. **المخرجات:** وثيقة متطلبات مبدئية (Requirement Spec) توضح نطاق المشروع، يوقعها العميل.

4. **المدة:** يُتوقع 1-2 أيام.

5. **خطة المشروع والتخطيط:**

6. **الإجراء:** إعداد وثيقة تحتوي على قائمة الميزات النهائية (القسم ٢) وجدول زمني تفصيلي (كالمخطط أعلاه).

7. **المخرجات:** جدول المراحل (Gantt) وخطة العمل المعتمدة من العميل.

8. **المدة:** حوالي أسبوع مع مراجعة العميل.

9. **تطوير مرحلي ومراجعات Sprint:**

10. **الإجراء:** تنفيذ الميزات على مراحل (مثلاً: المصادقة ثم CRUD ثم الكل)، مع جلسات مراجعة منتظمة مع العميل بعد كل مرحلتين.

11. **المخرجات المرحلية:** نسخ قابلة للتجريب (يمكن عبر Postman) لكل مجموعة من الميزات المنجزة، وملاحظات العميل للتحسين.

12. **التواصل:** تقارير تقدم دورية (سلاكات/إيميل) على الأقل أسبوعياً.

13. **اختبارات وتوثيق:**

14. **الإجراء:** تطوير اختبارات آلية للـ API وتشغيلها بانتظام. تفعيل Scribe لإنشاء الوثائق.

15. **المخرجات:** تقارير الاختبارات مكتملة (تعداد الاختبارات المنجزة)، وروابط الوثائق الأوتوماتيكية (HTML).

16. **التوقيت:** مع نهاية كل تطوير مرحلة رئيسية.

17. **النشر النهائي والقبول:**

18. **الإجراء:** تحضير بيئة الإنتاج وإطلاق النسخة النهائية. إجراء اختبار تحميل/أداء أخير.

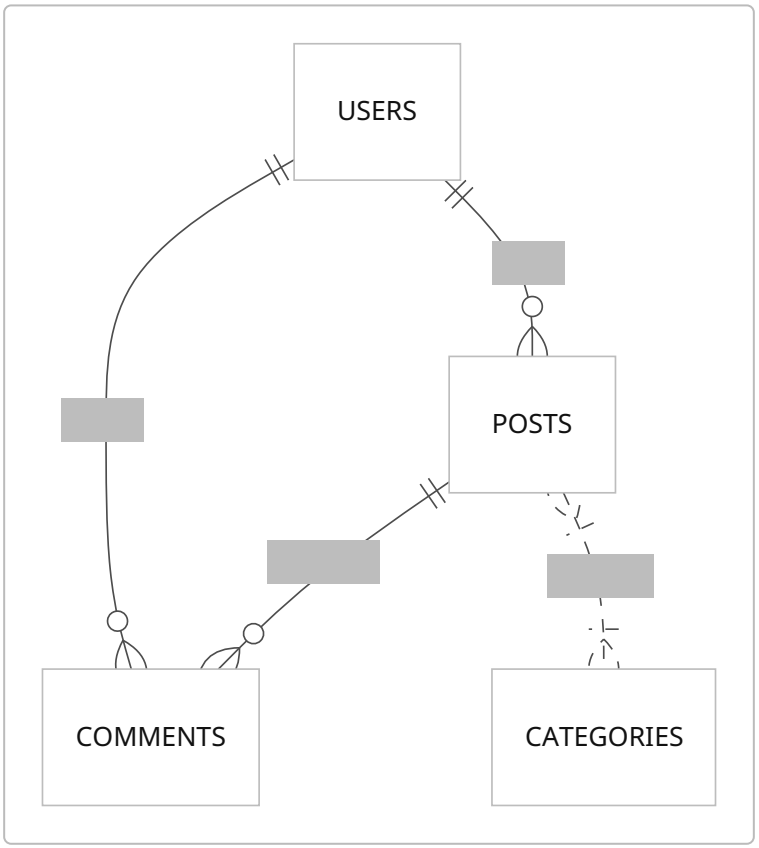
19. **المخرجات:** نشر الـ API على البيئة الحية (Production)، مع تسليم تقرير مختصر بالإنجازات والملاحظات النهائية.
20. **التسليم النهائي:** تقديم جميع الملفات المذكورة أعلاه للعميل.

ملاحظة: يُنصح بعقد اجتماع **مراجعة منتصف الطريق** (Mid-term Review) لمتابعة سير العمل وضبط التوقعات قبل المرحلة الأخيرة.

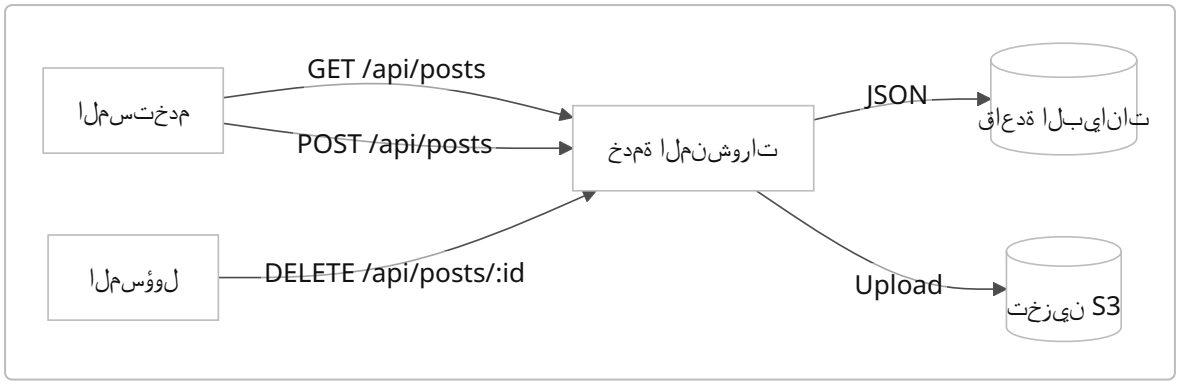
٨. جداول ومخططات مرئية

لضمان فهم بصري لنظام الـ API، نقترح تضمين المخططات التالية باستخدام **Mermaid**:

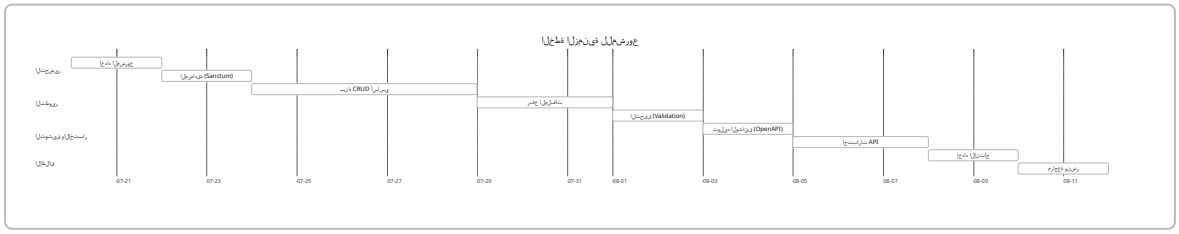
• **مخطط علاقات ER (Entity-Relationship):** يوضح الكيانات وعلاقاتها (مثال تخيلي):



• **مخطط تدفق نقاط نهاية الـ API (Flowchart):** يوضح كيفية تعامل المستخدمين مع الـ API (مثال):



• **الجدول الزمني (Gantt):** مثال مبسط لخطة المشروع (كما في القسم ٤):



هذه المخططات يمكن تحويلها إلى صور واستخدامها في ملف PDF النهائي لتوضيح البنية والجدولة بشكل مرئي.

٩. المصادر المفضلة

• **وثائق Laravel الرسمية:** معلومات حول الموارد (Resources) ونظام المصادقة Sanctum/Passport، التحقق، وسياسات الصلاحيات.
• **معيار OpenAPI (Swagger):** (مثلاً على swagger.io) لتنسيق المواصفات【L243-L245†32】.
• **وثائق Amazon S3:** لشرح الخيارات التخزينية والتوافر العالي لـ 【L75-L80†21】S3.
• **مستندات Scribe:** دليل حزمة Scribe لتوليد وثائق الـ API【L54-L58†12】.
• **مدونة Twilio (مقارنة Sanctum وPassport):** شرح خصائص المصادقتين【L962-L967†14】.
• **مقالات عن تصميم الـ API:** مثل قائمة التحقق غير الوظيفية (للأداء والأمان)【L55-L62†10】【L133-L140†10】.
• **مدونة Postman (الـ Collection format):** لشرح تنسيق ملفات Postman【L124-L131†30】.

ملاحظة: لا توجد مصادر عربية معروفة تغطي كل هذه التفاصيل بالأسلوب المطلوب؛ لذلك نركز على المصادر التقنية الإنجليزية الرسمية والمجتمعية.

١٠. صيغة الملف النهائي للتسليم وأدوات التحويل

نسلم الوثائق في عدة صيغ لتغطية احتياجات العميل والمطورين:

• **PDF نهائي:** المستند المكتمل (يتضمن النصوص والمخططات السابقة) بصيغة PDF بجودة عالية. بناءً على طلب العميل "صدرلي النتيجة باستخدام الأدوات المناسبة على شكل ملف PDF مع الرسومات"، سنقوم بإنشاء هذا الملف.
• **مستودع Markdown:** نسخة الـ Markdown من الوثيقة (مثلاً `report.md` في GitHub) تشمل الجداول والكود (بما في ذلك مخططات mermaid).
• **OpenAPI Spec (YAML/JSON):** الملف `openapi.yaml` (أو `openapi.json`).
• **Postman Collection:** الملف `collection.json` لمجموعة Postman.
• **صور المخططات:** ملفات `er_diagram.png` و`api_flowchart.png` و`gantt_chart.png` (مخرجات مخططات Mermaid).
• **دليل النشر:** `deployment_guide.pdf` أو `.md`.

• **اختبارات:** ملف أو مجلد `tests/` يحتوي على اختبارات PHPUnit.
• **توثيق Scribe:** ملفات HTML الناتجة من تشغيل Scribe (مثلاً `docs.html`).

أدوات تحويل Markdown إلى PDF:
- **Pandoc:** أداة قوية لتحويل Markdown إلى PDF (PDF، LaTeX). مثال:

```
pandoc report.md -o report.pdf
```

- **wkhtmltopdf:** يمكن أولاً توليد HTML من Markdown (Pandoc) أو مواقع مثل GitHub Pages) ثم تحويله:

```
pandoc report.md -o report.html
wkhtmltopdf report.html report.pdf
```

- **GitHub Actions:** بالإمكان إعداد Workflow على GitHub لتشغيل أوامر Pandoc/mermaid تلقائياً عند كل Push لتوليد PDF تلقائياً.
- **دمج مخططات Mermaid:** يمكن توليد الرسومات بصيغة PNG باستخدام mermaid-cli (`mmdc`)، ثم تضمينها في Markdown. على سبيل المثال:

```
mmdc -i diagrams/flow.mmd -o er_diagram.png
```

ثم يشمل Markdown الصيغة `![](er_diagram.png)` . Pandoc سيدمج الصور في الـ PDF الناتج.

ملحوظة عن تضمين الرموز: تأكد من تضمين أي صور Mermaid مصغرة بدقة كافية في PDF (أو استخدم SVG إن أمكن).

ملخص تسليم الملفات للعميل:

• `report.pdf` (التقرير النهائي بصيغة PDF مع المخططات)
• `report.md` (نسخة Markdown من المستند)
• `openapi.yaml` (ملف مواصفات الـ API)
• `postman_collection.json` (مجموعة Postman)
• `er_diagram.png` , `api_flowchart.png` , `gantt_chart.png` (مخططات ER وFlow وGantt)
• `deployment_guide.pdf` أو `.md` (دليل النشر)
• `tests/` (أو ملفات `TestSuite.php`)
• `docs.html` (وثائق Scribe HTML إن وجدت).

الافتراضات المفتوحة الأساسية: نذكر أن العميل لم يحدد بعض التفاصيل مثل نوع التكامل الخارجي أو منصات الحسابات الاجتماعية، لذا اعتمدنا السيناريوات الأكثر شيوعاً وحضرنا المجال للتوسع لاحقاً.

يرجى تأكيد الموافقة على الهيكل والخطة أعلاه قبل المضي في كتابة المستند النهائي وتصميم ملف PDF المطلوب.
